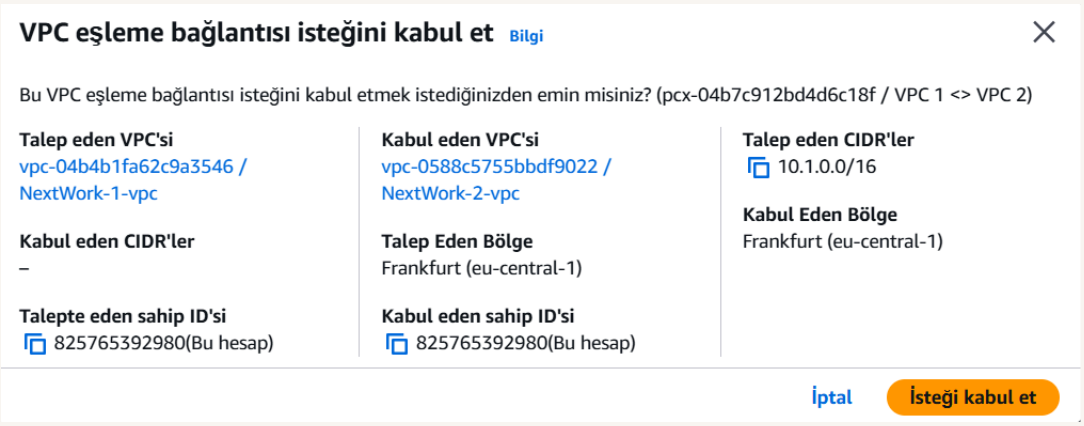
nextwork.ai

VPC Peering

P Pelin Ece Burgun



Pelin Ece Burgun
NextWork Student

Introducing Today's Project!

What is Amazon VPC?

Amazon VPC is a logically isolated section of the AWS cloud where you can launch AWS resources in a virtual network that you define. It is useful because it gives you full control over your network environment — including IP address ranges, subnets, route tables, and security rules — while keeping your resources isolated from other AWS customers by default. It also enables advanced architectures like VPC peering, private subnets, and VPN connections.

How I used Amazon VPC in this project

In this project, I created two Amazon VPCs — NextWork-1 (10.1.0.0/16) and NextWork-2 (10.2.0.0/16) — each with a public subnet and an internet gateway. I then connected them via a VPC peering connection and updated both route tables so traffic could flow between them. I also launched EC2 instances in each VPC and configured security groups to allow the right traffic. The entire project was essentially an exercise in building and validating a multi-VPC network architecture from scratch.

One thing I didn't expect in this project was...

I didn't expect that accepting a peering connection wouldn't be enough on its own — I assumed the VPCs would just start communicating. The fact that you also have to manually update route tables in both VPCs, and separately configure security group inbound rules, made it clear that AWS networking has multiple independent layers of control. Each layer (peering, routing, security groups) has to be explicitly configured — nothing is implicit.

Pelin Ece Burgun
NextWork Student

This project took me...

The project took me approximately 2 hours.

Pelin Ece Burgun
NextWork Student

In the first part of my project...

Step 1 - Set up my VPC

I started my project by launching two VPCs using the "VPC and more" wizard. The first VPC, NextWork-1, uses the CIDR block 10.1.0.0/16, and the second, NextWork-2, uses 10.2.0.0/16. Each VPC has one public subnet and an internet gateway. I chose non-overlapping CIDR blocks intentionally so that routing between them won't cause conflicts later.

Step 2 - Create a Peering Connection

In this step, I will create a peering connection between NextWork-1-VPC (Requester) and NextWork-2-VPC (Accepter), because the two VPCs need an explicit connection link before they can route traffic to each other.

Step 3 - Update Route Tables

In this step, I will update the route tables of both VPCs to direct traffic through the peering connection, because without explicit routes, neither VPC knows how to reach the other — the peering connection exists but traffic has no path to follow.

Step 4 - Launch EC2 Instances

In this step, I will launch one EC2 instance inside each VPC — Instance - NextWork VPC 1 and Instance - NextWork VPC 2 — because we need actual resources inside each VPC to test whether the peering connection allows traffic to flow between them. Without instances, there's nothing to send or receive test packets.

Pelin Ece Burgun
NextWork Student

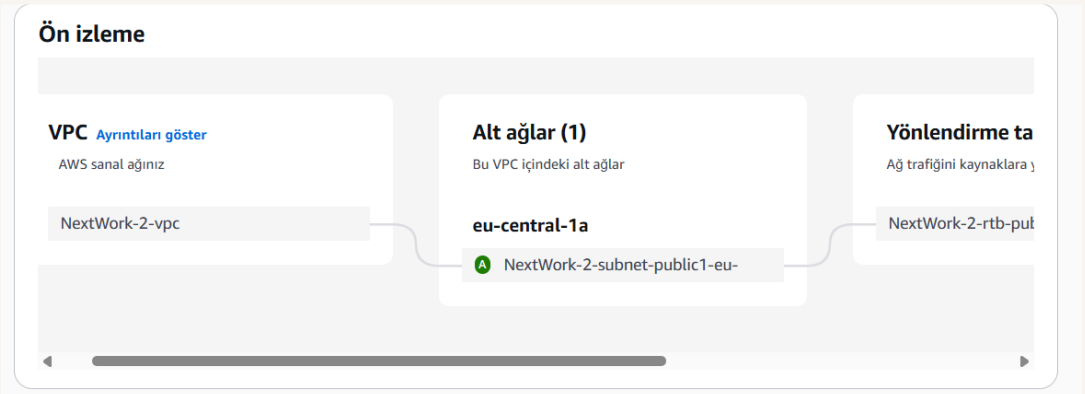
Multi-VPC Architecture

I launched two VPCs: NextWork-1 with CIDR block 10.1.0.0/16 and NextWork-2 with CIDR block 10.2.0.0/16. Each VPC was created with one public subnet, so I created 2 subnets in total. Each VPC also got an internet gateway and a route table automatically via the "VPC and more" wizard.

Each VPC needs a unique, non-overlapping CIDR block because when peering is established, AWS needs to route traffic unambiguously — if both VPCs shared the same IP range, the route tables would have conflicting destinations and couldn't determine which VPC a packet belongs to. We saw this directly when we accidentally entered VPC 1's CIDR block into VPC 1's own route table and got an error.

I also launched 2 EC2 instances

I didn't set up key pairs for these EC2 instances because we'll be using EC2 Instance Connect to access them. EC2 Instance Connect manages a temporary key pair on AWS's side automatically, so I don't need to create or store one myself. This simplifies the setup while still allowing secure SSH access.



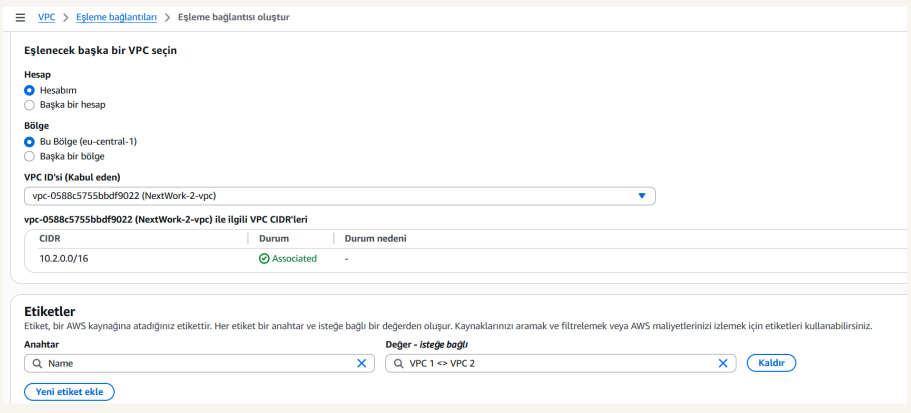
Pelin Ece Burgun
NextWork Student

VPC Peering

A VPC peering connection is a direct, private networking link between two VPCs that allows their resources to communicate using private IP addresses, without routing traffic through the public internet. The connection requires both sides to agree — one VPC sends a request and the other must accept it.

VPC peering connections exist because VPCs are isolated by design — they cannot communicate with each other by default. Peering provides a private, direct network link between two VPCs so their resources can exchange traffic using private IP addresses, without routing through the public internet. This is more secure, lower latency, and avoids data transfer costs associated with public routing.

The Requester is the VPC that initiates the peering connection — it sends an invitation to connect. In this project, that was NextWork-1-VPC. The Accepter is the VPC that receives the request and must explicitly approve it — here, NextWork-2-VPC. Both sides have to agree before the connection becomes active, which ensures no VPC gets peered without its "owner" consenting.

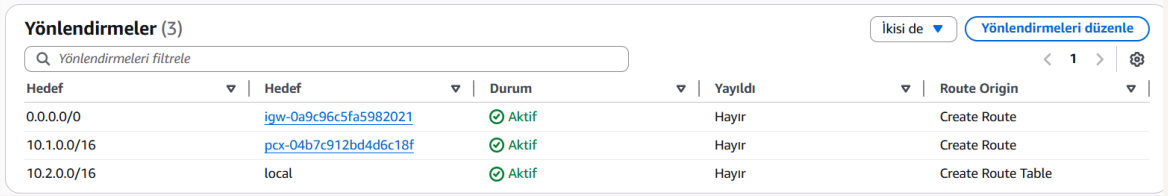


Pelin Ece Burgun
NextWork Student

Updating route tables

After accepting a peering connection, my VPCs' route tables need to be updated because a peering connection alone does not automatically tell traffic where to go. The route table is what actually instructs traffic destined for VPC 2's CIDR block (10.2.0.0/16) to use the peering connection, and vice versa. Without these routes, packets would have no path and would be dropped.

For VPC 1's route table, I added a route with destination 10.2.0.0/16 (VPC 2's CIDR block) pointing to the VPC 1 <> VPC 2 peering connection as the target. For VPC 2's route table, I added the mirror route: destination 10.1.0.0/16 pointing to the same peering connection. These two routes together create a bidirectional path between the VPCs.



In the second part of my project...

Step 5 - Use EC2 Instance Connect

In this step, I will attempt to connect to Instance - NextWork VPC 1 using EC2 Instance Connect, because I need terminal access to Instance 1 to run ping commands toward Instance 2 later. I expect to encounter a connection error and will need to troubleshoot it.

Step 6 - Connect to EC2 Instance 1

In this step, I will attempt EC2 Instance Connect again after resolving the missing public IP issue, and troubleshoot a second error — this time caused by the security group blocking inbound SSH traffic.

Step 7 - Test VPC Peering

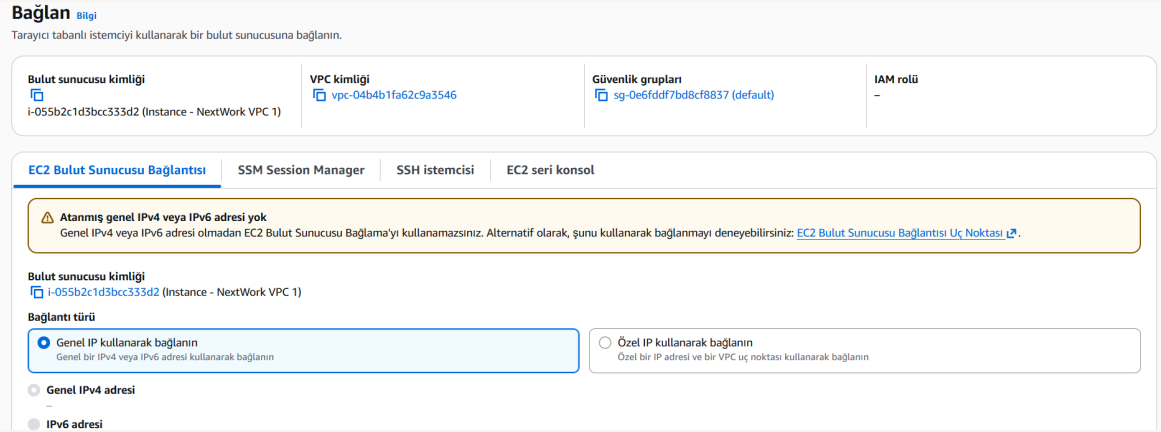
In this step, I will use the ping command from Instance 1's terminal to send ICMP test messages to Instance 2's private IP address, because ping is the most direct way to verify whether the peering connection actually routes traffic end-to-end. I expect to hit errors and will troubleshoot them by investigating VPC 2's security group inbound rules.

Pelin Ece Burgun
NextWork Student

Troubleshooting Instance Connect

I used EC2 Instance Connect to access Instance - NextWork VPC 1 directly from the browser without needing a locally stored key pair. However, the instance initially had no public IP address, which blocked the connection — EC2 Instance Connect routes over the public internet and requires a public IPv4. I resolved this by allocating an Elastic IP and associating it with the instance.

There were two layered errors. First, EC2 Instance Connect failed because the instance had no public IPv4 address — I had left Auto-assign public IP disabled during launch. EC2 Instance Connect routes over the public internet and requires a public IP to reach the instance. Second, after associating an Elastic IP, the connection still failed because the default security group only allowed inbound traffic from within the VPC itself, blocking SSH from the internet on port 22.

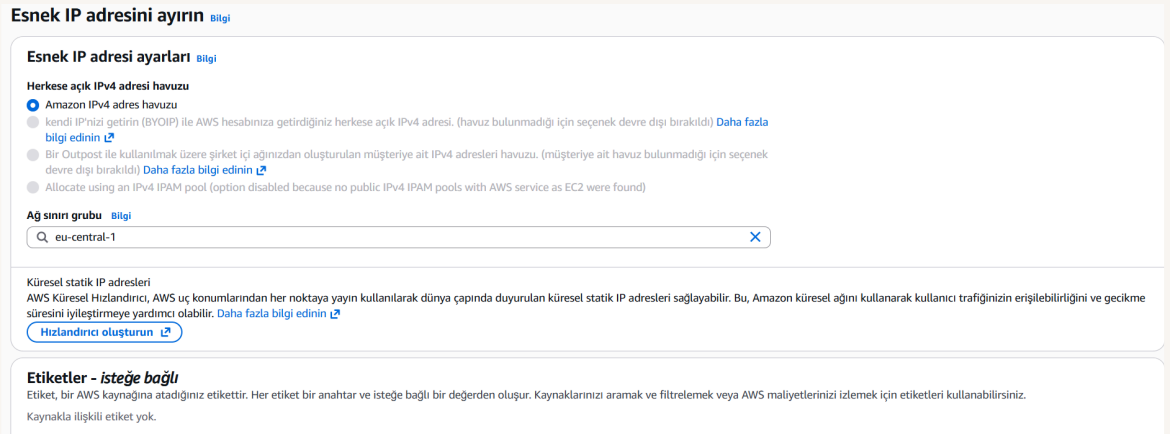


Pelin Ece Burgun
NextWork Student

Elastic IP addresses

Elastic IPs are static public IPv4 addresses that you allocate to your AWS account and can associate with EC2 instances. Unlike the default dynamic public IPs that change every time an instance restarts, an Elastic IP stays constant. This makes them useful both for keeping applications reachable at a fixed address and — as in this project — for assigning a public IP to an instance that was launched without one.

Since Instance - NextWork VPC 1 was launched without a public IP, EC2 Instance Connect had no way to reach it over the internet. Allocating an Elastic IP and associating it with the instance gave it a stable, reachable public IPv4 address. Once the IP was in place — and after also opening port 22 in the security group — EC2 Instance Connect could successfully establish the SSH session.

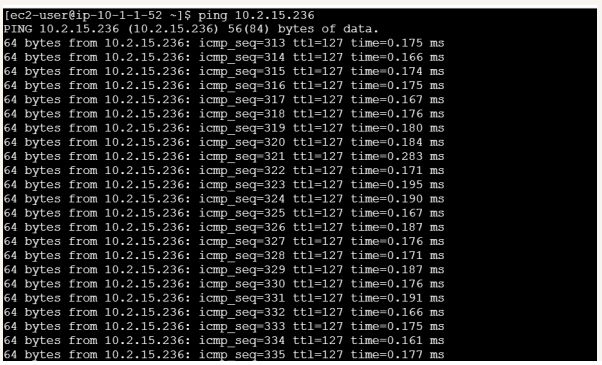


Troubleshooting ping issues

To test VPC peering, I ran the command ping <private IPv4 of Instance - NextWork VPC 2> from Instance 1's terminal via EC2 Instance Connect. Initially there were no replies, which indicated ICMP traffic was being blocked by VPC 2's security group. After adding an inbound rule to allow All ICMP - IPv4 from 10.1.0.0/16, the ping replies came through successfully, confirming the peering connection was working.

Ping sends ICMP packets from Instance 1 to Instance 2's private IP address. If the peering connection, route tables, and security groups are all correctly configured, Instance 2 will receive the packets and send replies back. Getting those replies confirms that private traffic is successfully crossing the VPC boundary through the peering connection — not through the public internet. No reply means something in that chain is broken, which forces you to systematically diagnose each layer.

I navigated to the VPC console, found VPC 2's default security group by filtering with its VPC ID, and selected "Edit inbound rules." I added a new rule with Type set to All ICMP - IPv4 and the Source set to 10.1.0.0/16 — VPC 1's CIDR block. This allows only ICMP traffic originating from VPC 1, rather than opening the instance to all traffic, which keeps the security footprint minimal while enabling the ping test to succeed.



The place to learn & showcase your skills

Check out nextwork.ai for more projects